Calibrated Offline-to-Online Reinforcement Learning with Minari: A Reproduction and Humanoid Stress Test

Anirudhan Kasthuri Matr. no.: 5973244
Sumit Kumar Patel Matr. no.: 5992625
Dinku Brammesh Matr. no.: 5987353

University of Freiburg Deep Learning Lab
August 2026

Abstract

Calibrated Q-Learning (Cal-QL) modifies Conservative Q-Learning (CQL) so that conservative value estimates are not pushed below a reference policy's value, addressing slow offline-to-online fine-tuning caused by miscalibration [1]. We adapted the public JAX implementation to Minari MuJoCo datasets, reproduced a matched CQL comparison on three locomotion tasks, and extended the study to Humanoid with online-only, replay-mixing, critic-capacity, and safety-cost ablations. The main result is from the Humanoid learning curves: several Cal-QL variants entered the high-return region earlier than CQL, and the 512–512 critic ablation reached return 7500 in about 18% fewer epochs. Terminal averages were more mixed: three-seed Cal-QL returns were 6046 ± 2236 on HalfCheetah, 2518 ± 1008 on Hopper, 4012 ± 1092 on Walker2d, and 7676 ± 659 on Humanoid, with Cal-QL only beating the single CQL seed on HalfCheetah. We additionally report two unsuccessful safety-cost ablations, with diagnosis in Sec. 4.2. The main limitations are that Appendix D is not fully reproduced and our code uses Monte Carlo return-to-go bounds rather than Appendix D's fitted SARSA Q-function [1].

1 Problem and Objective

Offline reinforcement learning learns from a fixed dataset $\mathcal{D}$ generated by a behavior policy μ, while offline-to-online learning subsequently collects transitions with the improving policy π. CQL combats out-of-distribution overestimation by learning conservative action values, but excessive pessimism can make the initial online policy appear worse than the behavior policy and delay improvement. Cal-QL introduces a reference-value lower bound to preserve conservatism without this particular miscalibration [1, 2].

Our objectives were to (i) port the JAX code from legacy D4RL data loading to Minari, (ii) run a controlled Cal-QL/CQL locomotion study, (iii) test transfer to the higher-dimensional Humanoid task, and (iv) produce a reproducible submission package with explicit limitations. D4RL originally standardized datasets and normalized-score conventions for offline RL [3]; our experiments instead report native, undiscounted Minari evaluation returns and therefore do not label them D4RL-normalized scores.

2 Cal-QL Formulation

We use the paper's discounted notation, with $Q^\pi(s,a)$ for action values and $V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q^\pi(s,a)]$ [1]. CQL learns conservative Q-values by penalizing policy actions relative to dataset actions while still fitting Bellman targets [2]. The part that matters for our reproduction is the Cal-QL calibration bound.

Cal-QL defines calibration relative to the behavior policy by requiring [1]

$$\mathbb{E}_{a\sim\pi}[Q^\pi_\theta(s,a)] \geq \mathbb{E}_{a\sim\mu}[Q^\mu(s,a)] = V^\mu(s), \quad \forall s \in \mathcal{D}, \tag{1}$$

and replaces CQL's conservative regularizer with [1]

$$\mathcal{R}_{\text{Cal}}(\theta) = \mathbb{E}_{s\sim\mathcal{D},a\sim\pi}[\max(Q_\theta(s,a), V^\mu(s))] - \mathbb{E}_{s,a\sim\mathcal{D}}Q_\theta(s,a). \tag{2}$$

The maximum masks the conservative push-down whenever a sampled policy action is already valued below the reference. The paper reports that simple return-to-go regression is adequate for terminal domains, while Appendix D fits a SARSA Q-function on nonterminating D4RL locomotion data [1].

3 Reproduction Method

3.1 Implementation and protocol

We kept the original CQL/Cal-QL update path and changed the data side to Minari's episode API. Each locomotion job used about one million offline gradient updates followed by 250,000 online environment steps; full commands, package versions, and hardware details are in our DLL notes. Cal-QL used three seeds, the resource-matched CQL baseline used one seed, and Humanoid added online-only SAC, mixing-ratio, critic-capacity, combined, and safety-cost ablations.

The submitted implementation computes a discounted Monte Carlo return-to-go for each stored trajectory,

$$\hat{G}_t = r_t + \gamma(1-d_t)\hat{G}_{t+1}, \qquad \hat{G}_T = 0, \tag{3}$$

and applies $\max(Q_\theta, \hat{G}_t)$ inside the Cal-QL conservative term. This is directly verified in `JaxCQL/replay_buffer.py`, `minari_compat.py`, and `conservative_sac.py`.

3.2 Appendix D and SARSA deviation

Appendix D reports D4RL locomotion results and states that, because those datasets do not terminate, the reference value was estimated by fitting a neural-network SARSA Q-function to the offline data [1]. However, it does not provide a complete locomotion-specific recipe sufficient for exact replication: the fitted SARSA model parameters or checkpoint, architecture and optimizer details, precise per-task training schedule, and full evaluation protocol are not specified. Therefore, we did not reproduce every Appendix D dataset, baseline, or cumulative-regret table.

Our public implementation also does not expose the paper's fitted SARSA reference model or its parameters. We therefore use the trajectory Monte Carlo estimator in Eq. 3. This choice is consistent with the paper's general terminal-task procedure but is *not* a faithful replacement for Appendix D's SARSA estimator on time-limited, nonterminal locomotion data [1]. It can bias the bound near artificial episode cutoffs, so we treat our results as a Minari adaptation with stated deviations rather than an exact Appendix D reproduction.

4 Results

Table 1 reports the last archived evaluation value. Each value is the final logged `evaluation/average_return`, where one evaluation point is averaged over evaluation rollouts; it is not an average over the whole learning curve. The Cal-QL uncertainty is the sample standard deviation across three seeds; the CQL entries are single runs and therefore have no uncertainty estimate.

Table 1: Terminal native Minari returns. Higher is better.

Task	Cal-QL (3 seeds)	CQL (seed 0)
HalfCheetah	6046 ± 2236	5800
Hopper	2518 ± 1008	3570
Walker2d	4012 ± 1092	6137
Humanoid	7676 ± 659	7862

Cal-QL was 4.2% above the CQL seed on HalfCheetah, but 29.5% below on Hopper and 34.6% below on Walker2d. These are terminal values only, so we do not use this table alone to make a claim about convergence speed. For Humanoid, Cal-QL was close to the CQL seed in final score, but the available curves below show faster offline-to-online transfer for the Cal-QL variants. The safest conclusion is that offline-pretrained CQL-family agents strongly exceeded the online-only SAC control (Table 2).

4.1 Positive Humanoid ablations

Table 2: Humanoid last-evaluation returns for controls and positive ablations.

Setting	Last eval. return
Cal-QL, standard (3 seeds)	7676 ± 659
CQL (seed 0)	7862
Online-only SAC (seed 0)	2272
Cal-QL, mix $= 0.25$ (seed 0)	6721
Cal-QL, critic 512–512 (seed 0)	7870
Cal-QL, critic 512–512 + mix $= 0.25$ (seed 0)	7963

The terminal table is useful, but it hides the thing we cared about most: how quickly the policy improves after offline pretraining. Figure 1 uses training epoch on the x-axis. The 512–512 critic was the cleanest positive ablation: it reached return 7500 at epoch 1357, compared with epoch 1661 for the CQL seed, so it needed about 18% fewer epochs to enter the high-return region and still finished at 7870. The mix $= 0.25$ run reached 7500 at epoch 1411 and looked fairly steady after the jump, but it finished lower at 6721, so the offline data may also have kept the policy closer to medium-quality behavior. Combining the two ideas was not simply additive: critic 512–512 plus mix $= 0.25$ finished high at 7963, but it crossed 7500 later than the critic-only run and had larger drops during online fine-tuning.

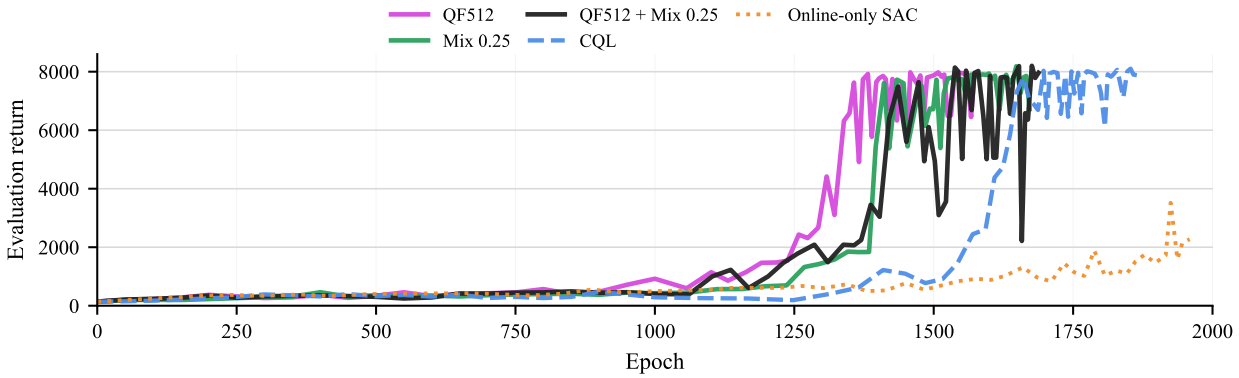

Figure 1: Humanoid evaluation return over training epoch. Epoch 1000 marks the switch from offline pretraining to online fine-tuning.

4.2 Negative safety-cost ablations

We also tested a safety-motivated extension where falling was treated as an additional cost. This was our own exploratory change within the Cal-QL reproduction, although the general idea is related to constrained RL, Lagrangian reward penalties, and learned safety critics [4, 5, 6]. With $c_t = \mathbf{1}\{\text{terminated}\}$, the reduced constrained problem is

$$\max_{\pi} J_r(\pi) \quad \text{s.t.} \quad J_c(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma_c^t c_t\right] \le d. \tag{4}$$

We tried two simpler implementations of this idea. The first was a fixed fall penalty:

$$\max_{\pi} \ \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t (r_t - \beta c_t)\right], \qquad \beta = 100. \tag{5}$$

The second learned a separate fall-cost critic and used it inside the actor loss:

$$Q^\pi_{\text{fall}}(s,a) = \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma_c^t c_t \,\middle|\, s_0 = s, a_0 = a\right],$$
$$\mathcal{L}_\pi = \mathbb{E}_{s,a\sim\pi}\left[\alpha \log \pi(a|s) - Q_r(s,a) + \lambda Q_{\text{fall}}(s,a)\right], \qquad \lambda = 10. \tag{6}$$

Figure 2 shows the first 3000 epochs of the safety-cost runs. Neither variant helped the main thing we wanted from Cal-QL, which was faster offline-to-online transfer. The fixed fall penalty stayed in the low-return region for a long time and crossed 7500 only at epoch 2447, much later than the 512–512 critic baseline. The learned $Q_{\text{fall}}(s,a)$ critic also improved late, crossing 7500 at epoch 2703, but its path was very noisy. So in this window the safety ideas were not useless, but they were negative ablations for convergence speed and stability.

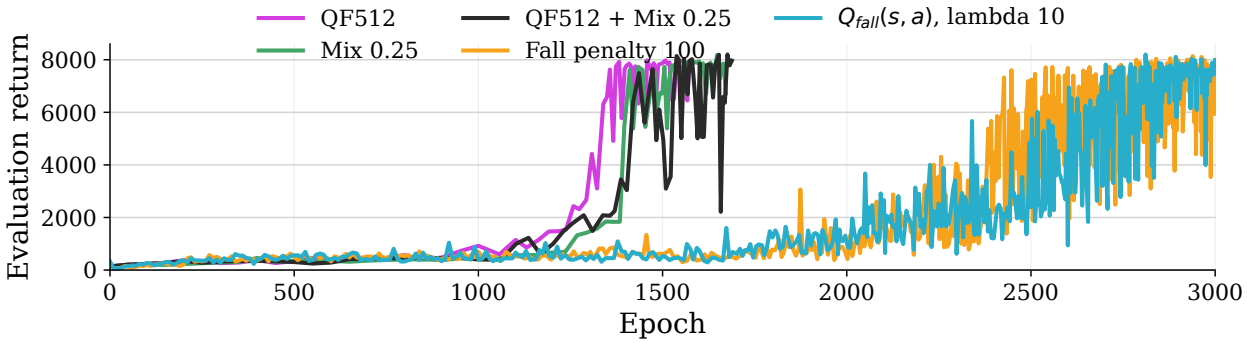

Figure 2: Exploratory safety-cost ablations on Humanoid, shown up to epoch 3000 with the positive Cal-QL ablation baselines.

Our interpretation is that the optimization was harder than the formulation looked. The fall signal is sparse and delayed, because the positive cost appears only near termination. A fixed penalty therefore changes mostly the last transition of an episode, and if the weight is large it also changes the scale of the Monte Carlo return-to-go values used by Cal-QL. The learned cost critic has an extra credit-assignment problem: most sampled transitions have zero cost, but the actor still has

to balance reward values against predicted future fall cost. Since our implementation used a fixed weighted actor penalty rather than a full constrained-RL solver, the result depended strongly on the chosen λ. To judge this direction properly, it needs a more comprehensive study, closer to dedicated safety-RL work, because reward scale, cost scale, and the constraint weight all interact. With more project time, we would first try reward/cost normalization before combining the reward critic and Q_{fall}, then sweep β and λ instead of choosing one value. We would also decouple the fall critic from the main critic, for example with a separate learning rate or more balanced samples around termination. We treat these as negative ablations of our implementation, not as proof that safety constraints are a bad idea.

5 Limitations and Conclusions

The submitted experiment lists contain 19 completed full-budget runs. For this report, we also used three additional Humanoid ablation runs exported from W&B: QF512+Mix, FallPenalty100, and Q_{fall}. This gives 22 completed runs in total. Some comparisons are still uneven because Cal-QL has more seeds than CQL in several settings, and because the mixing ratios are not identical across every run. We also do not have final-window averages for the locomotion runs, and some old run metadata could not be fully recovered. Native Minari returns prevent direct numeric comparison with the paper's D4RL-normalized tables. There is also a possible MuJoCo environment-version mismatch: Minari recovers current Gymnasium/MuJoCo environments for online fine-tuning, while the original D4RL setup used older task versions, so strict v4/v5-style comparability is not guaranteed. We did not separately verify the Humanoid action-coordinate mapping, and time-limit truncations are stored as replay terminals; fixing either issue would require a new controlled rerun. We keep these limitations in our DLL notes rather than changing them after the runs were finished.

Overall, the study shows that the Minari port works and that offline pretraining transfers strongly to Humanoid relative to online-only SAC. It does not show that Cal-QL consistently beats CQL in final return. The more interesting Humanoid result is about speed: several Cal-QL variants entered the high-return region earlier, which matches the motivation of Cal-QL better than the last evaluation value alone. A paper-faithful locomotion test still requires the Appendix D SARSA reference and a fully specified training/evaluation protocol.

6 Team Work Distribution

We split the work by function, but the experimental work was shared: each of us owned one locomotion reproduction task, and all three of us helped with at least part of the Humanoid ablation matrix.

- **Anirudhan Kasthuri—experiments and infrastructure:** set up WSL/Linux, MuJoCo, JAX/CUDA, and RTX 4060 runs; ran reproduction and ablation jobs and tracked convergence.
- **Sumit Kumar Patel—implementation and ablations:** adapted the Minari code path; proposed the safety-cost ablations; ran reproduction and Humanoid ablation jobs; prepared W&B plots.
- **Dinku Brammesh—reproduction and ablations:** ran reproduction and ablation jobs; reviewed tables, references, and figure consistency.

References

[1] M. Nakamoto, Y. Zhai, A. Singh, M. S. Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine, "Cal-QL: Calibrated Offline RL Pre-Training for Efficient Online Fine-Tuning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[2] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-Learning for Offline Reinforcement Learning," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[3] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4RL: Datasets for Deep Data-Driven Reinforcement Learning," *arXiv:2004.07219*, 2020.

[4] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained Policy Optimization," *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70:22–31, 2017.

[5] C. Tessler, D. J. Mankowitz, and S. Mannor, "Reward Constrained Policy Optimization," *International Conference on Learning Representations*, 2019.

[6] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg, "Recovery RL: Safe Reinforcement Learning with Learned Recovery Zones," *IEEE Robotics and Automation Letters and International Conference on Robotics and Automation*, 2021.